



CS 498:

Basics for GenAI

— Transformers

Fan Lai

The Grainger College of Engineering

Materials with thanks to Minjia Zhang, Arvind Krishnamurthy,
Jimmy Shong, and many colleagues

- **Team registration**
 - 4-5 members in a team by Sept 5

⋮

Preferred Papers/Dates for Joining the Panel Discussion (e.g., 09/26) *

Long answer text

- **Transformers and FlashAttention**
 - Why we need transformers? (Advantage)
 - How does transformer work?
 - How to run it efficiently?

Transformers Have Revolutionized the Field of ML



- Originally developed to solve machine translation
- Underpins virtually every state-of-the-art NLP and computer vision model

 Microsoft +  OpenAI

 Google +  DeepMind

 TESLA +  xAI

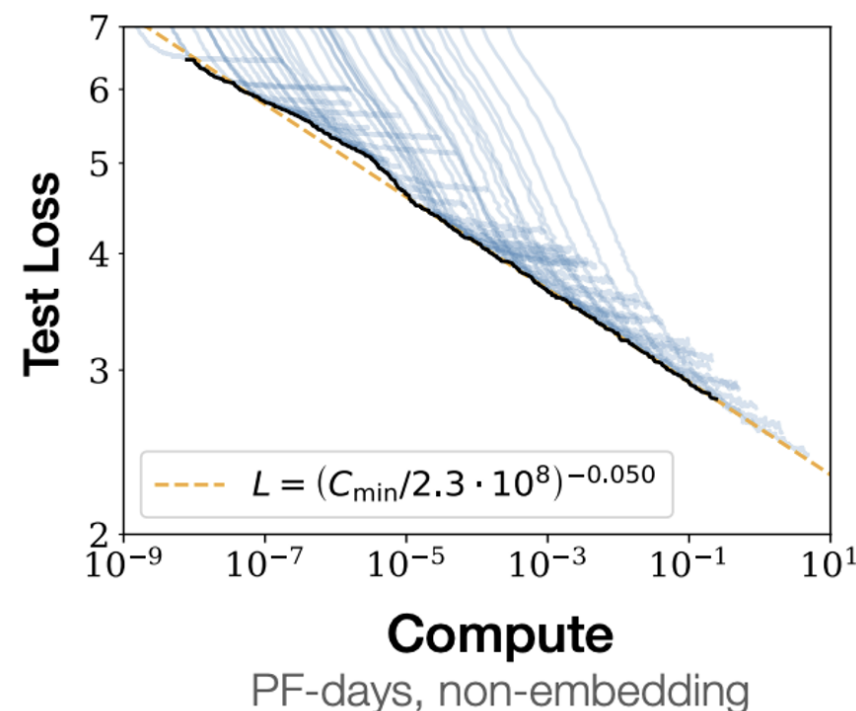
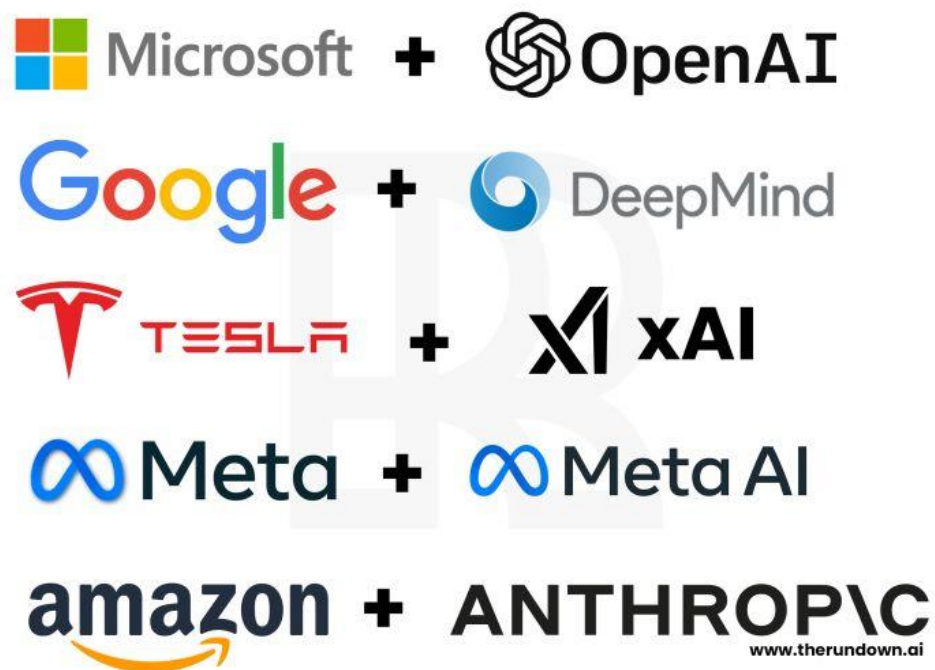
 Meta +  Meta AI

 amazon + ANTHROPIC
www.therundown.ai

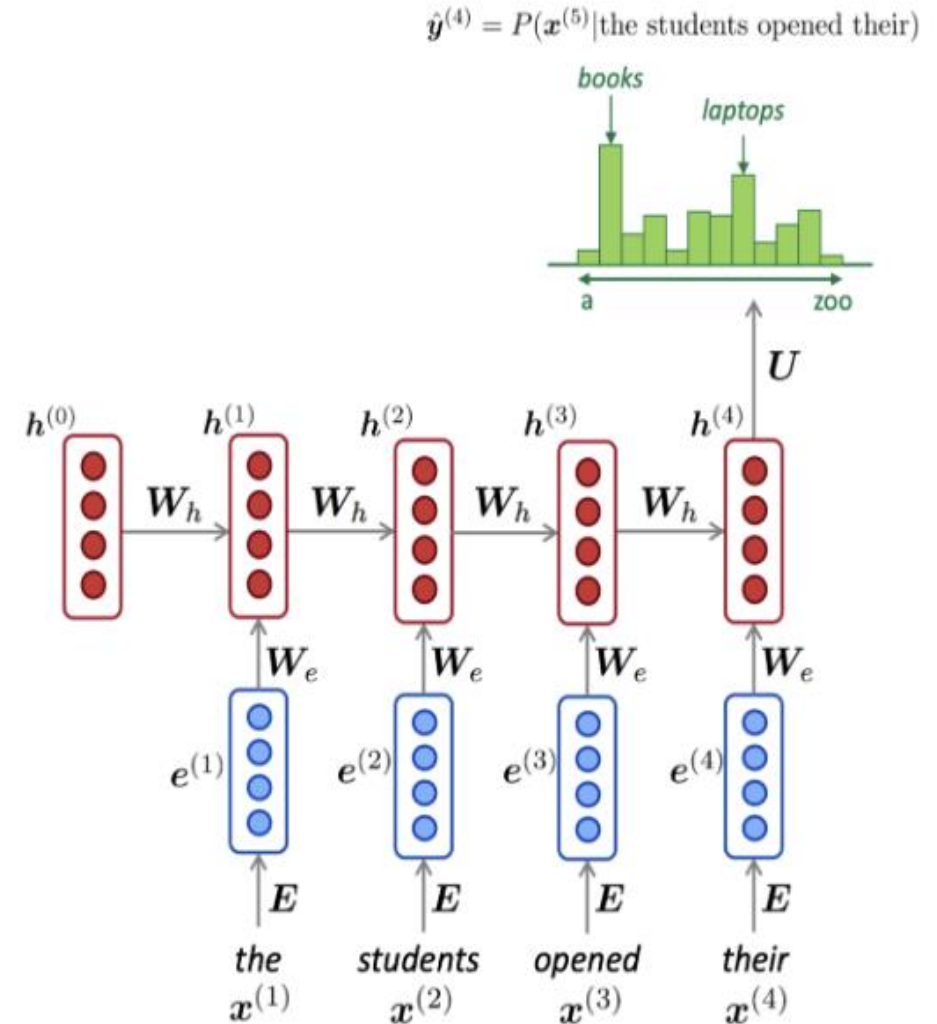
Transformers Have Revolutionized the Field of ML



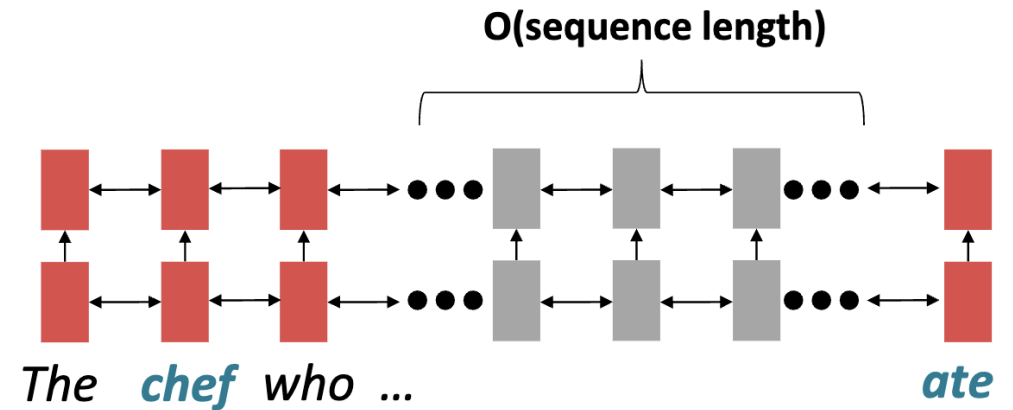
- Originally developed to solve machine translation
- Underpins virtually every state-of-the-art NLP and computer vision model
- Modeling performance improves as we increase model size, training data, and compute



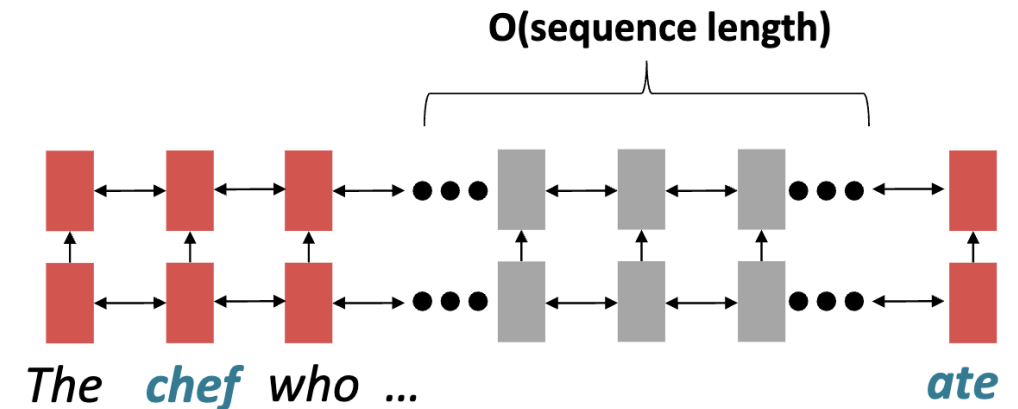
- The de facto strategy in NLP was to encode sentences with an Recurrent Neural Network (RNN)
- Define your text output as a sequence and use an RNN to generate it.



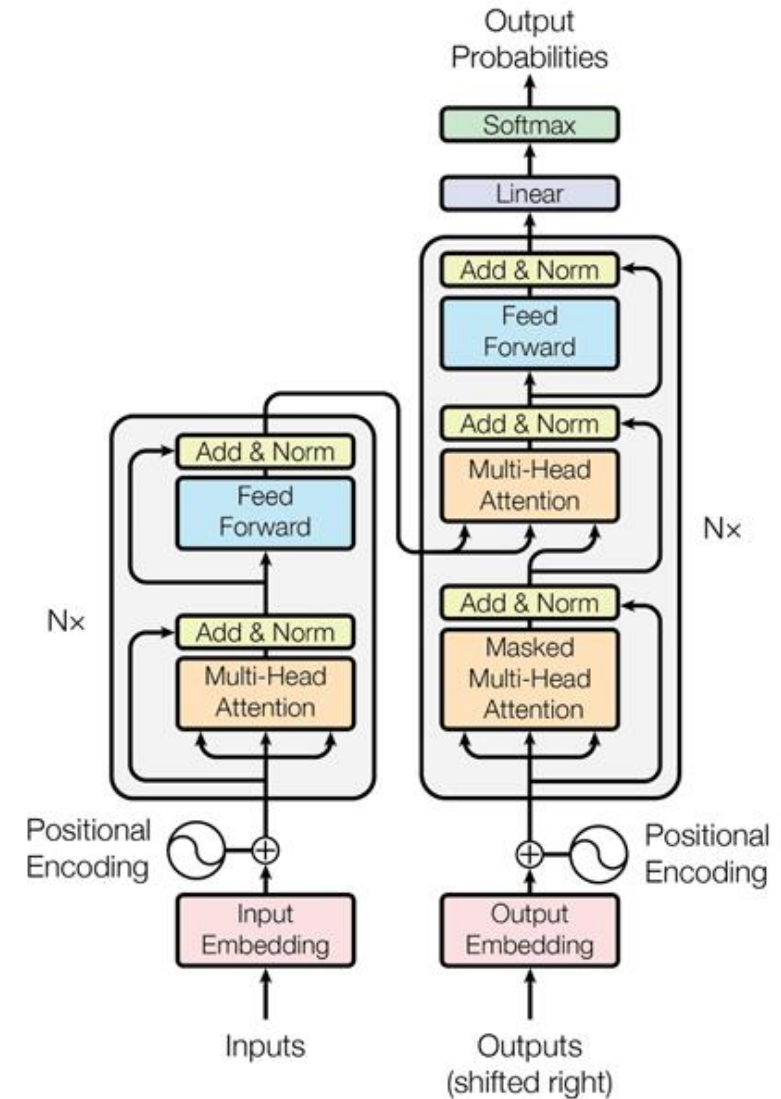
- Sequential computation prevents parallelization
- Difficult to access information from many steps back due vanishing/ exploding gradients

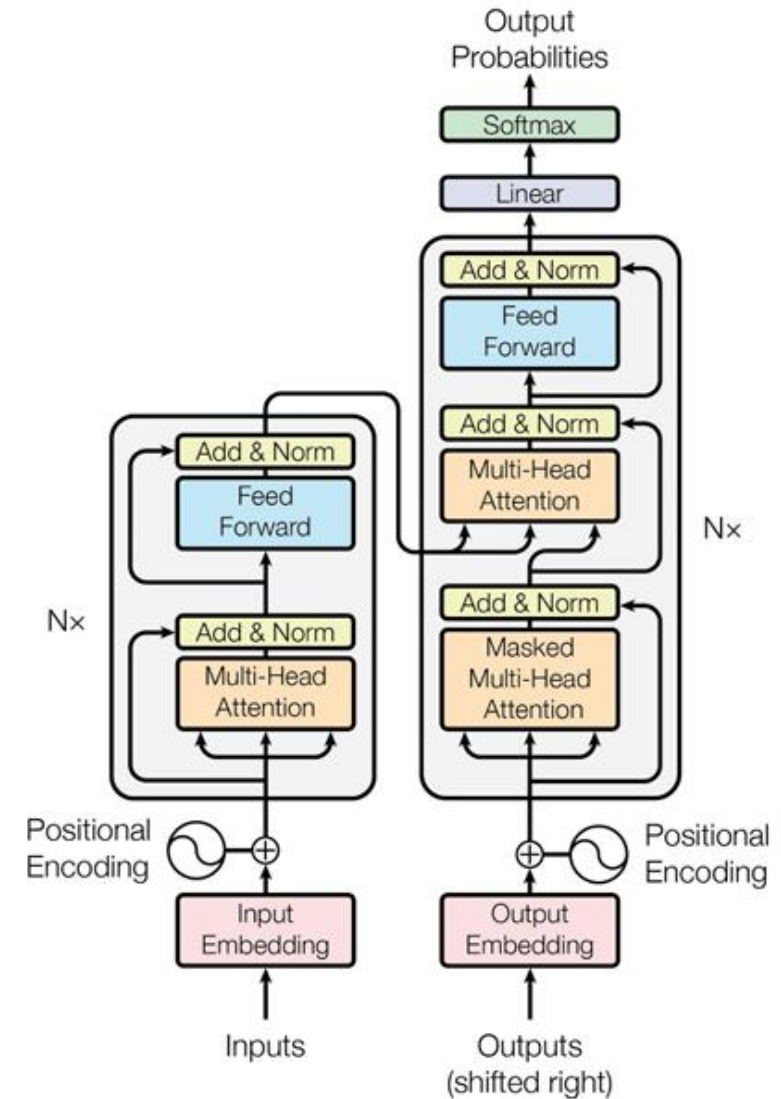
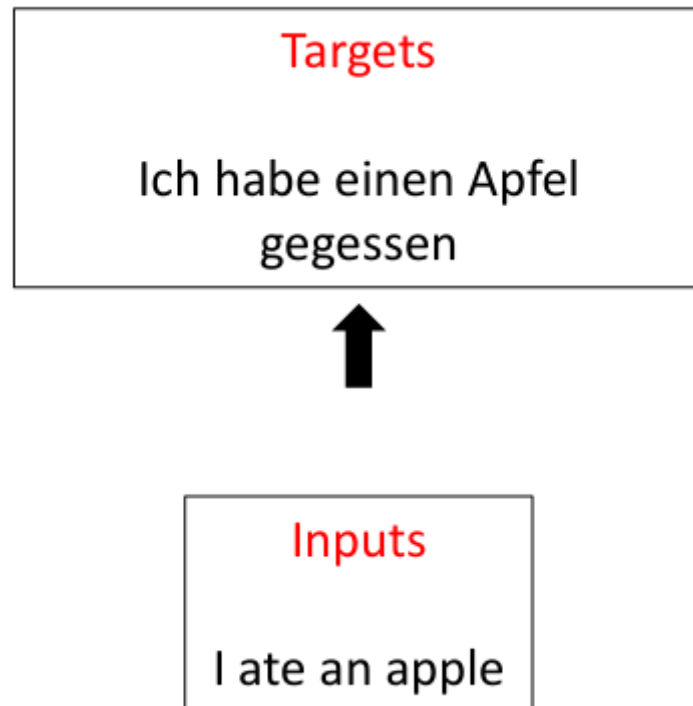


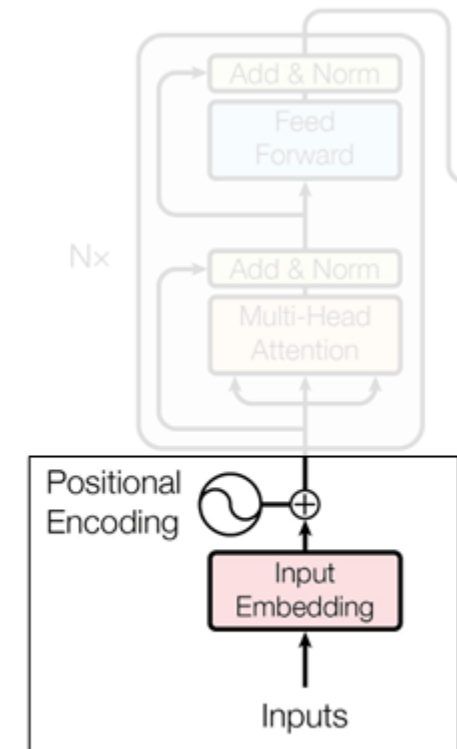
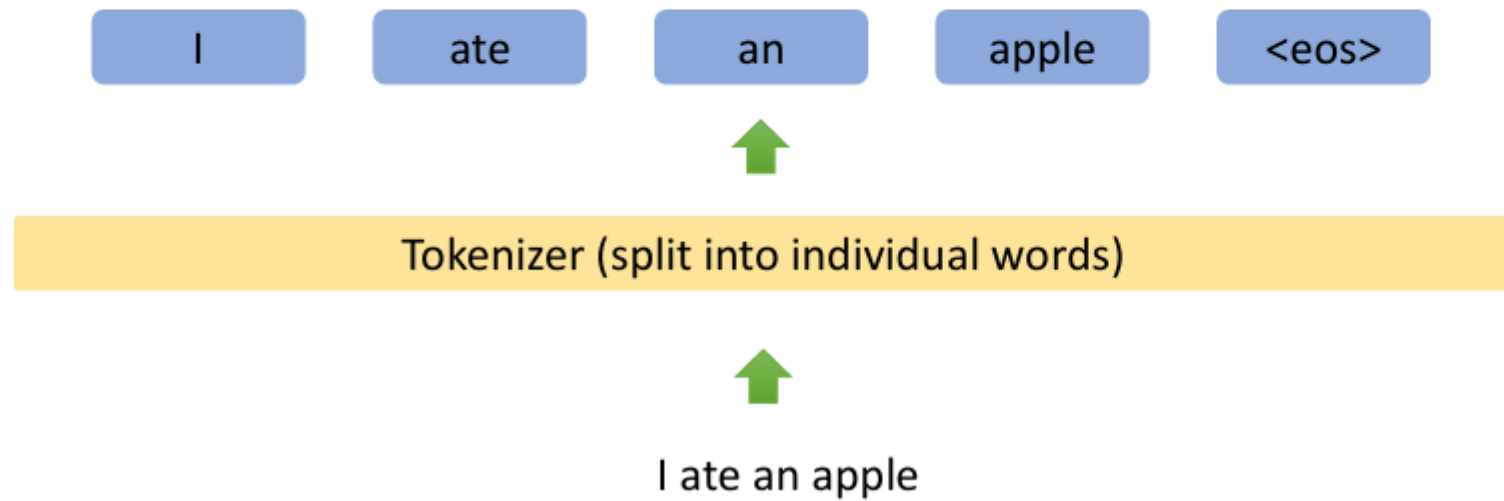
- Sequential computation prevents parallelization
- Difficult to access information from many steps back due vanishing/ exploding gradients
- We want to do the following:
 - Maximize the amount of computation that can be parallelized
 - Do not increase computational complexity per layer while avoiding vanishing gradients



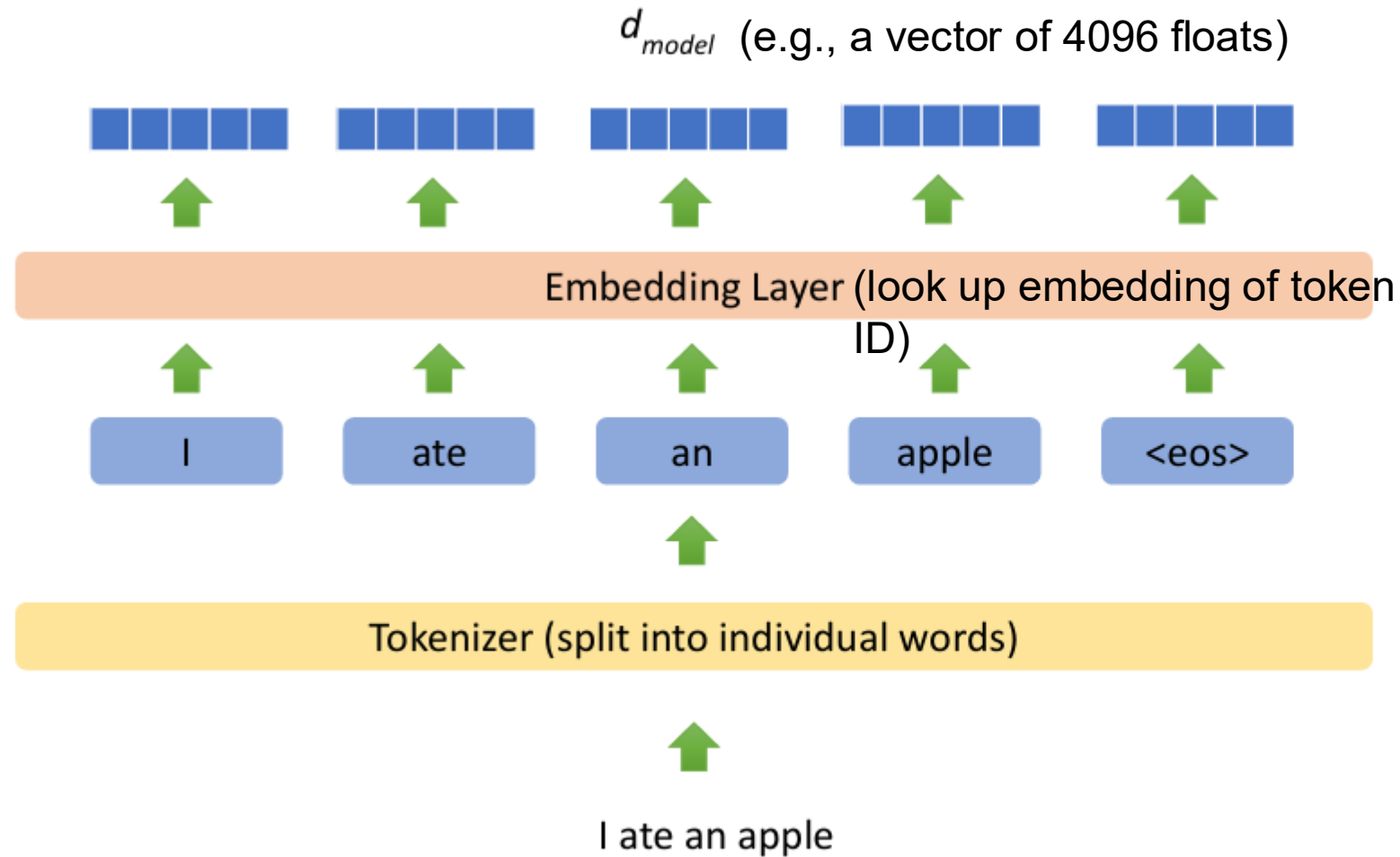
- Tokenization
- Input embeddings
- Position encodings
- Query, Key, Value
- Self-attention
- Multi-head attention
- Feed forward
- Add & norm
- Residual
- Masked attention
- Causal attention
- Linear
- Softmax
- Encoders
- Decoder
- Encoder-decoder



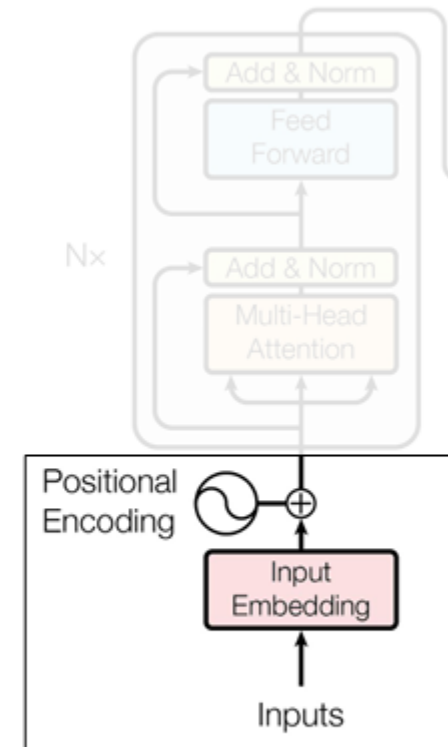


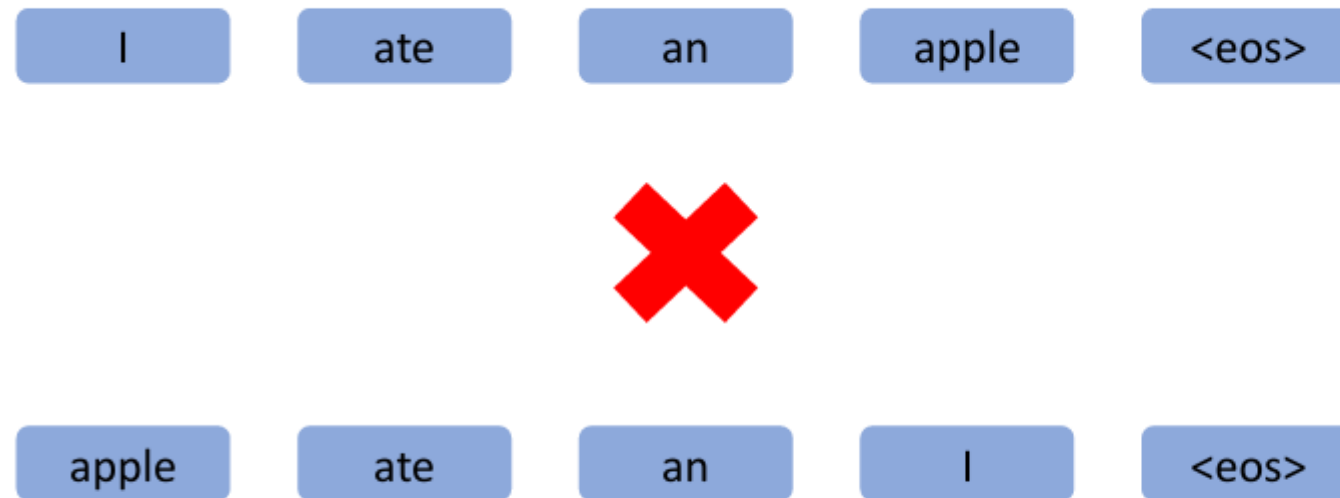


Input Embeddings

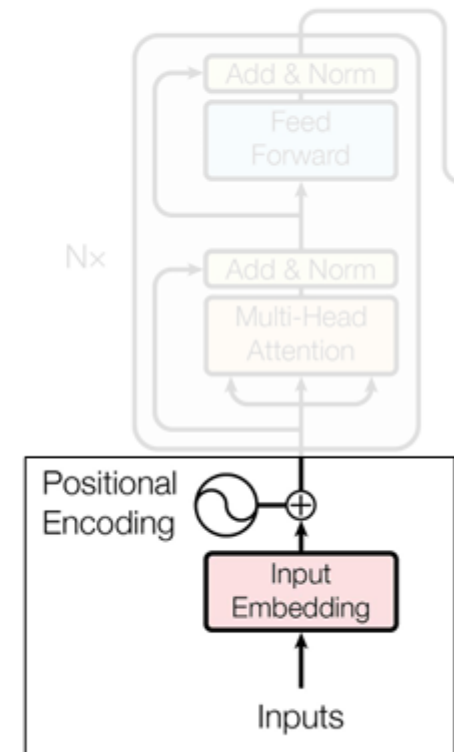


Generate Input Embeddings





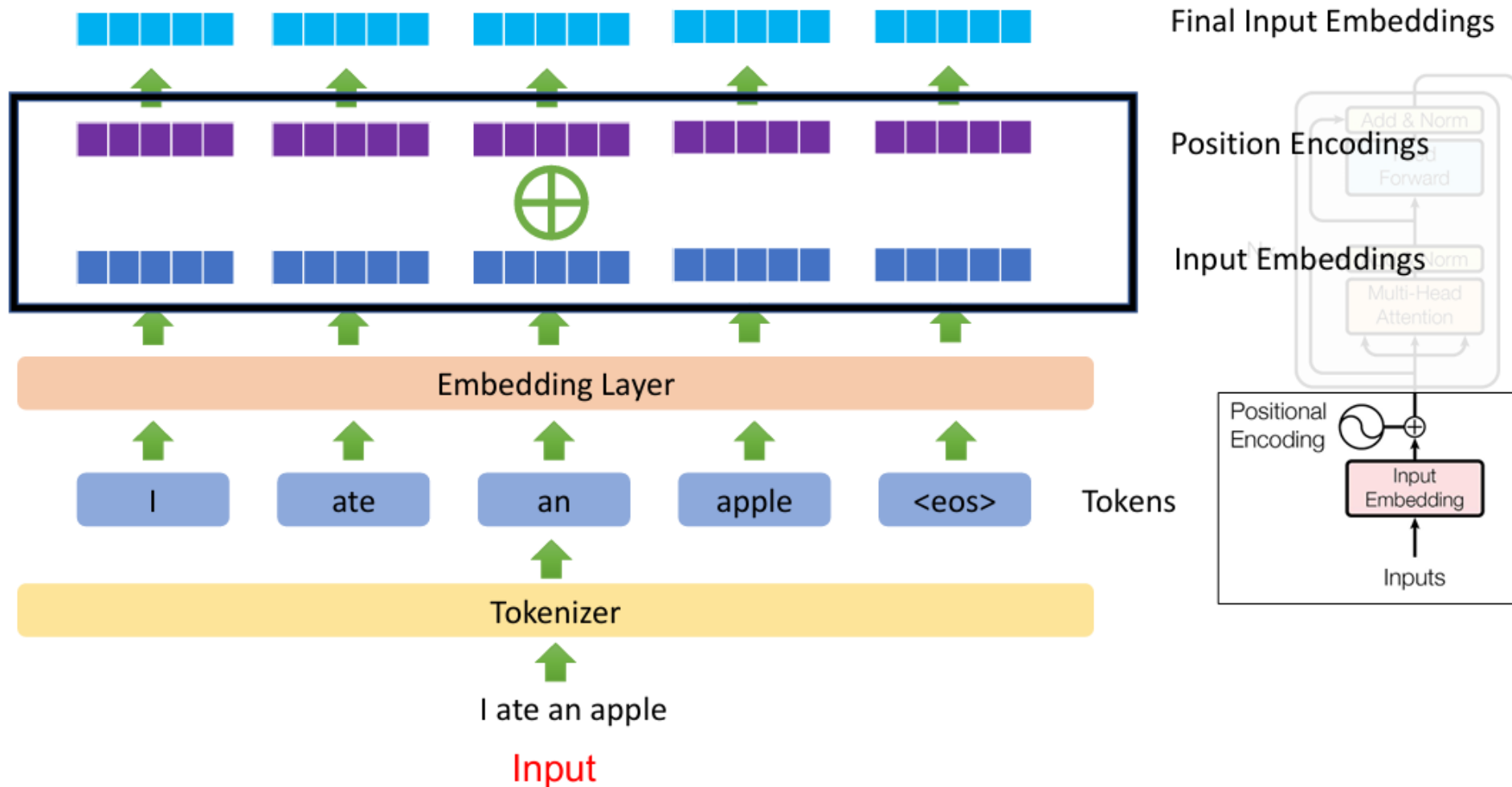
- The position of words in a sentence carries information!
- **Idea:** Add some information to the representation at the beginning that indicates where it is in the sentence



Positional Encoding



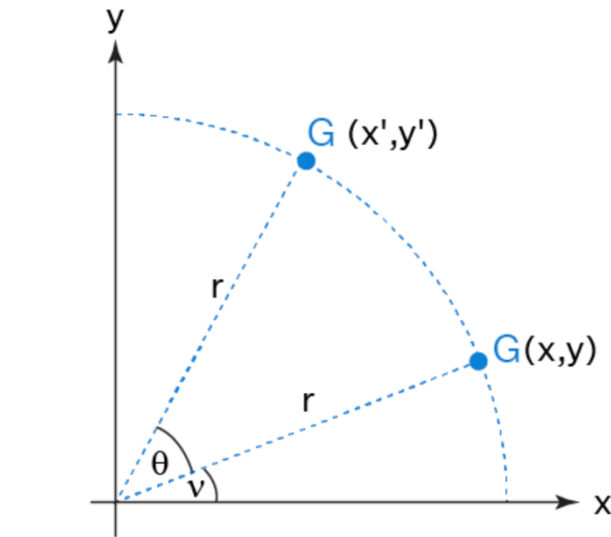
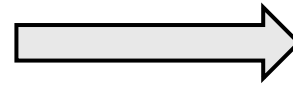
Absolute positional encoding (original Transformer): The token embeddings and the absolute position embedding are **added** together element-wise.



- Most recent LLMs adopt RoPE: incorporate relative positional information by rotating a token's embedding by its position

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}.$$

Thus, $\begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$ will be the rotation matrix.



<https://www.cuemath.com/algebra/rotation-matrix/>

[1] RoFormer: Enhanced Transformer with Rotary Position Embedding, 2021

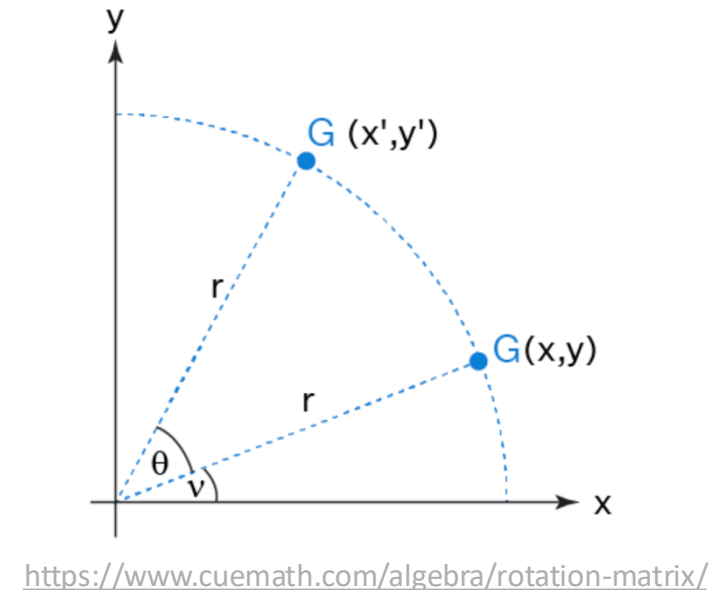
- Most recent LLMs adopt RoPE: incorporate relative positional information by rotating a token's embedding by its position

$$R_{\Theta,p}^d q = \begin{pmatrix} q_1 \\ q_2 \\ q_3 \\ q_4 \\ \vdots \\ q_{d-1} \\ q_d \end{pmatrix} \otimes \begin{pmatrix} \cos p\theta_1 \\ \cos p\theta_1 \\ \cos p\theta_2 \\ \cos p\theta_2 \\ \vdots \\ \cos p\theta_{d/2} \\ \cos p\theta_{d/2} \end{pmatrix} + \begin{pmatrix} -q_2 \\ q_1 \\ -q_4 \\ q_3 \\ \vdots \\ -q_d \\ q_{d-1} \end{pmatrix} \otimes \begin{pmatrix} \sin p\theta_1 \\ \sin p\theta_1 \\ \sin p\theta_2 \\ \sin p\theta_2 \\ \vdots \\ \sin p\theta_{d/2} \\ \sin p\theta_{d/2} \end{pmatrix}$$

token's position (p)

Query vector
(e.g., d=1024 floats)

Pre-determined rotation degrees
(e.g., $\theta_1=1$, $\theta_2=0.01$)



[1] RoFormer: Enhanced Transformer with Rotary Position Embedding, 2021

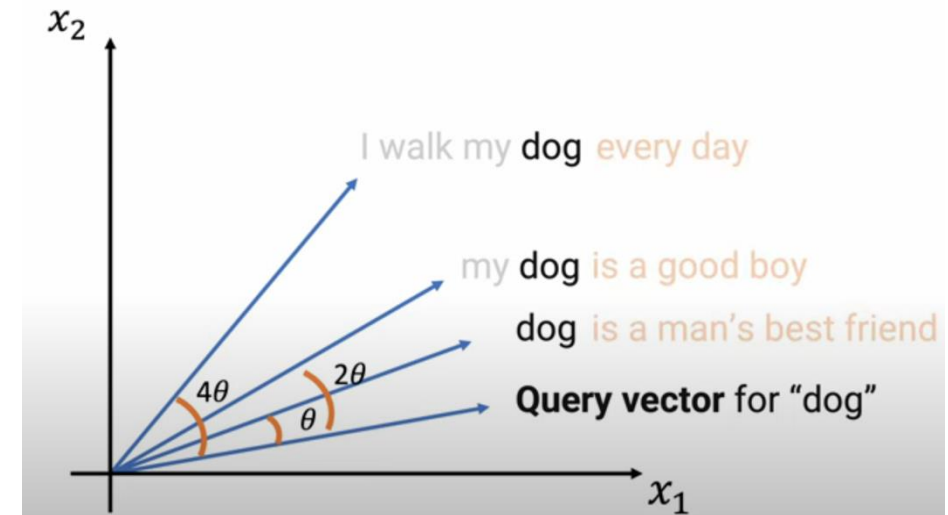
- Most recent LLMs adopt RoPE: incorporate relative positional information by rotating a token's embedding by its position

$$R_{\Theta,p}^d q = \begin{pmatrix} q_1 \\ q_2 \\ q_3 \\ q_4 \\ \vdots \\ q_{d-1} \\ q_d \end{pmatrix} \otimes \begin{pmatrix} \cos p\theta_1 \\ \sin p\theta_1 \\ \cos p\theta_2 \\ \sin p\theta_2 \\ \vdots \\ \cos p\theta_{d/2} \\ \sin p\theta_{d/2} \end{pmatrix} + \begin{pmatrix} -q_2 \\ q_1 \\ -q_4 \\ q_3 \\ \vdots \\ -q_d \\ q_{d-1} \end{pmatrix} \otimes \begin{pmatrix} \sin p\theta_1 \\ \cos p\theta_1 \\ \sin p\theta_2 \\ \cos p\theta_2 \\ \vdots \\ \sin p\theta_{d/2} \\ \cos p\theta_{d/2} \end{pmatrix}$$

token's position (p)

Query vector
(e.g., d=1024 floats)

Pre-determined rotation degrees
(e.g., $\theta_1=1$, $\theta_2=0.01$)



[1] RoFormer: Enhanced Transformer with Rotary Position Embedding, 2021

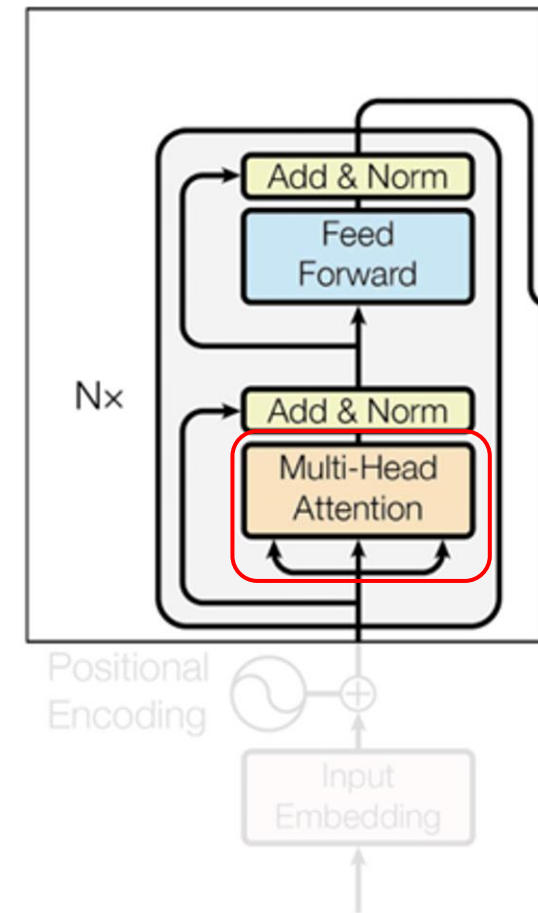
- Most recent LLMs adopt RoPE: incorporate relative positional information by rotating a token's embedding by its position

$$R_{\Theta,p}^d q = \begin{pmatrix} q_1 \\ q_2 \\ q_3 \\ q_4 \\ \vdots \\ q_{d-1} \\ q_d \end{pmatrix} \otimes \begin{pmatrix} \cos p\theta_1 \\ \sin p\theta_1 \\ \cos p\theta_2 \\ \sin p\theta_2 \\ \vdots \\ \cos p\theta_{d/2} \\ \sin p\theta_{d/2} \end{pmatrix} + \begin{pmatrix} -q_2 \\ q_1 \\ -q_4 \\ q_3 \\ \vdots \\ -q_d \\ q_{d-1} \end{pmatrix} \otimes \begin{pmatrix} \sin p\theta_1 \\ \cos p\theta_1 \\ \sin p\theta_2 \\ \cos p\theta_2 \\ \vdots \\ \sin p\theta_{d/2} \\ \cos p\theta_{d/2} \end{pmatrix}$$

token's position (p)

Query vector
(e.g., d=1024 floats)

Pre-determined rotation degrees
(e.g., $\theta_1=1$, $\theta_2=0.01$)

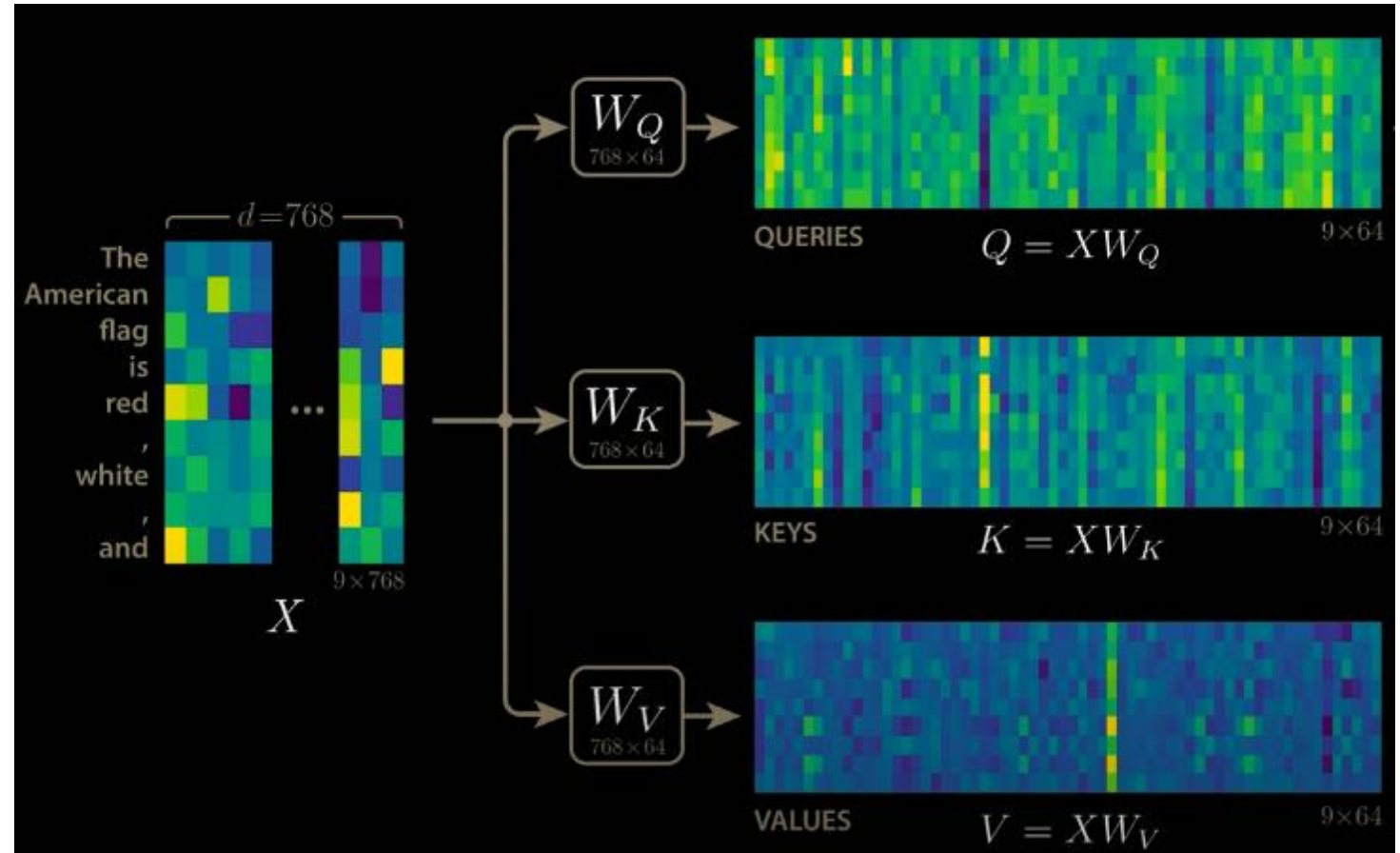


[1] RoFormer: Enhanced Transformer with Rotary Position Embedding, 2021

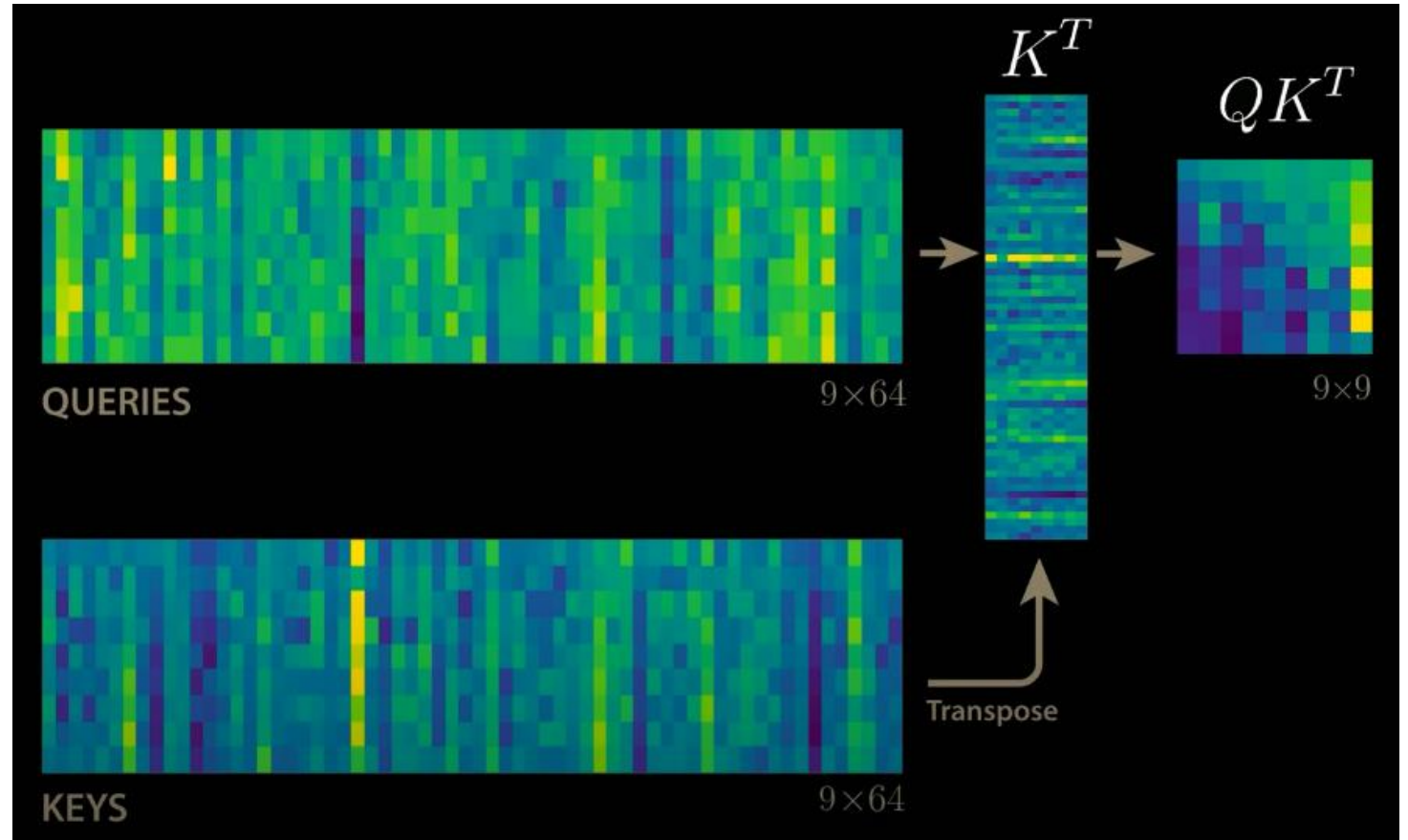
Self-Attention: #1



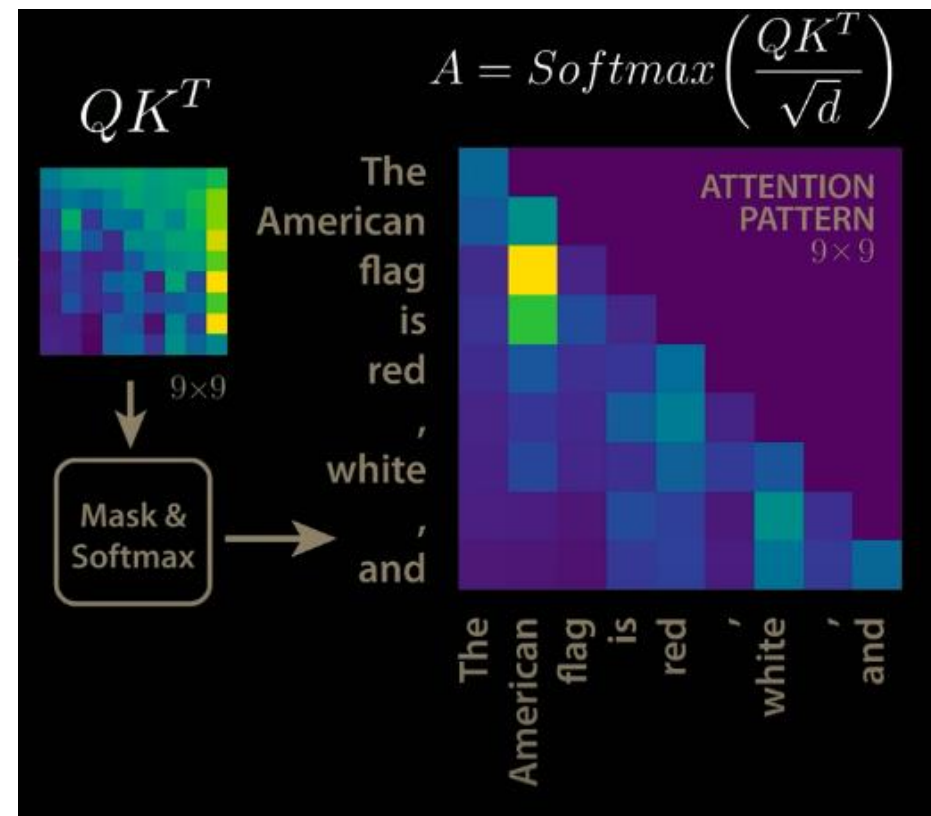
- Step 1: Create three matrix from input matrix (X)
 - Query Vector (Q)
 - Key Vector (K)
 - Value Vector (V)
- These are created by multiplying x_i with **trained weight** matrices W^Q, W^K, W^V

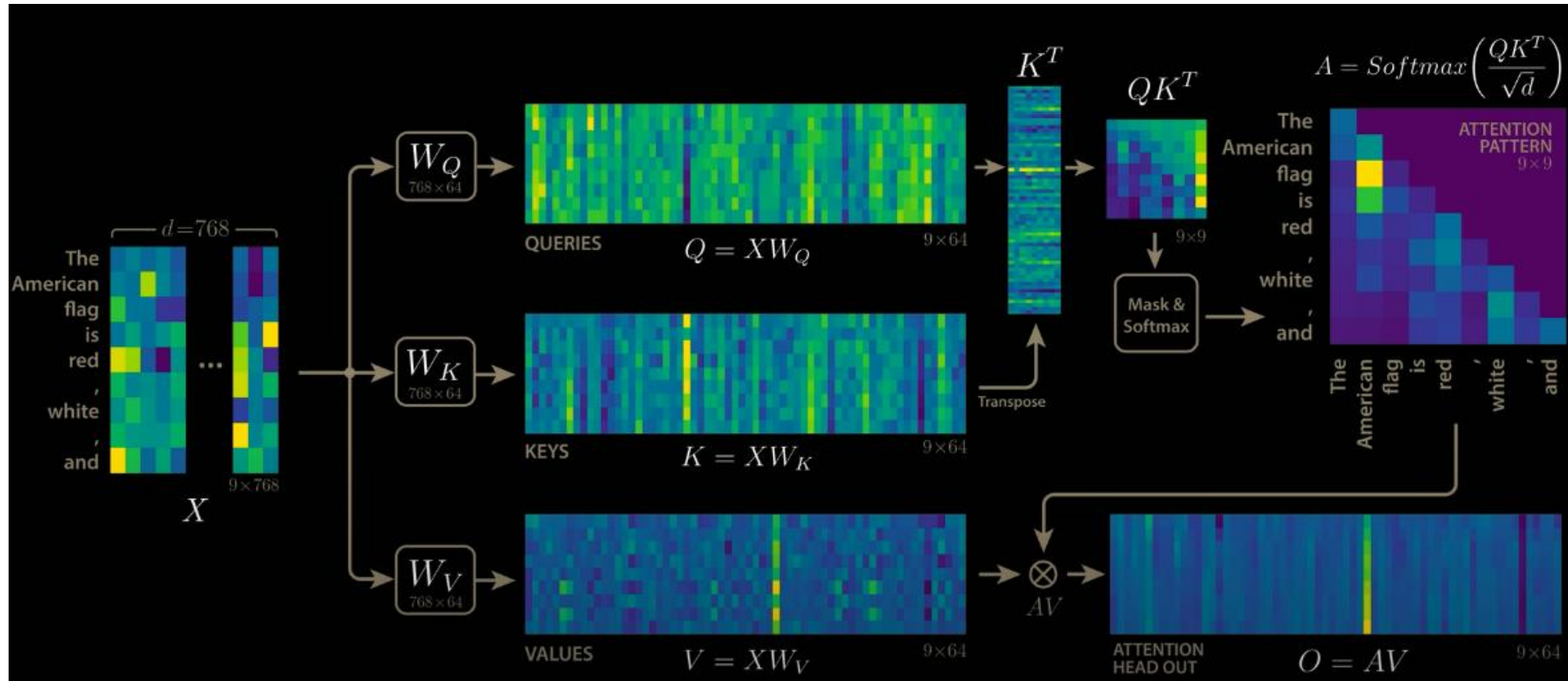


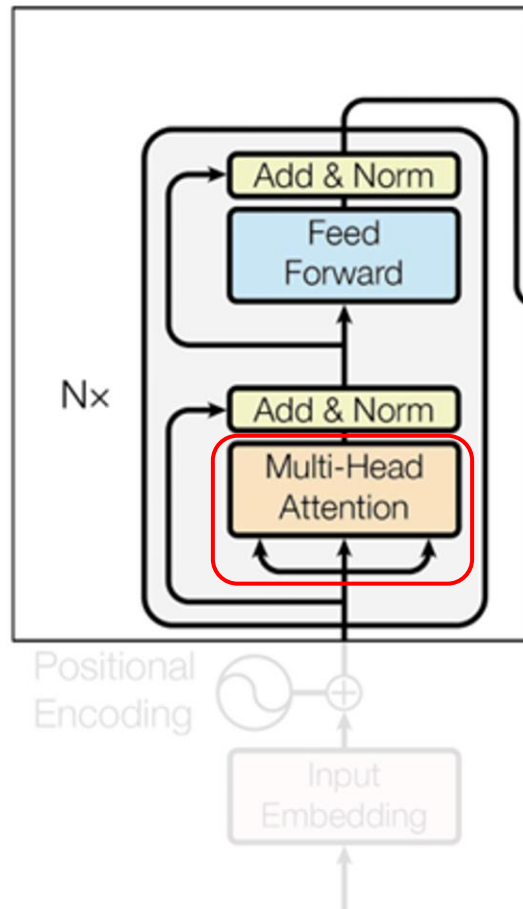
- Step 1: Create three matrix from input matrix (X)
 - Query Vector (Q)
 - Key Vector (K)
 - Value Vector (V)
- Step 2: Calculate self-attention scores - score all tokens of input sentence against themselves



- Step 1: Create three matrix from input matrix (X)
 - Query Vector (Q)
 - Key Vector (K)
 - Value Vector (V)
- Step 2: Calculate self-attention scores - score all tokens of input sentence against themselves
- Step 3: Scores then divided by $\sqrt{d}$ for stable gradients. Use Softmax to normalize scores







Which word in our vocabulary
is associated with this index?

Get the index of the cell
with the highest value
(argmax)

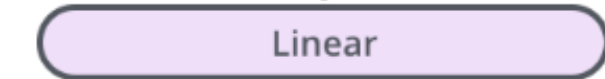
log_probs



logits



Decoder stack output



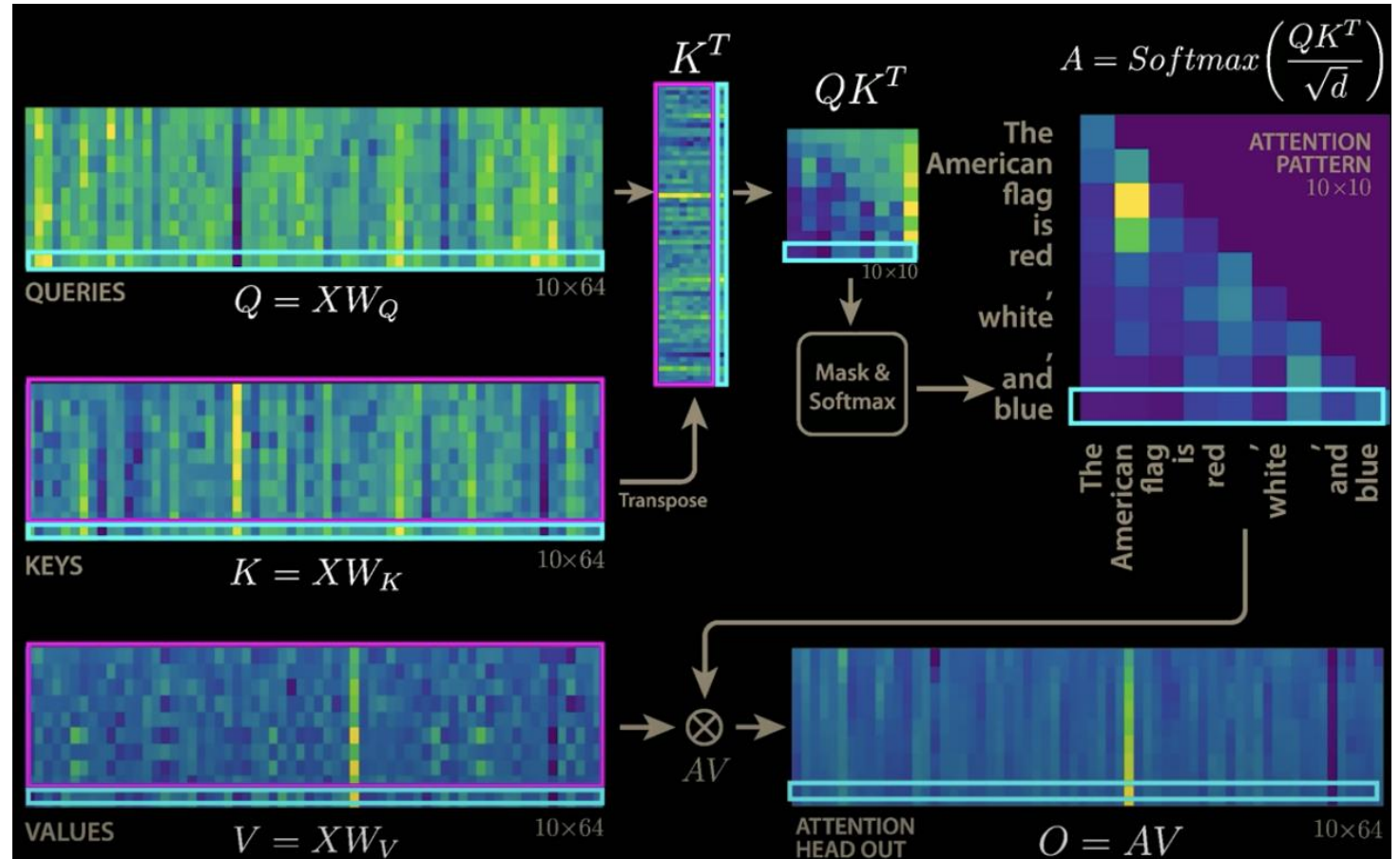
Step 5: preceding layers feed subsequent
layers over transformer blocks

Step 6: The Final Linear and Softmax Layer

Autoregressive Generation Introduces KV Ccache



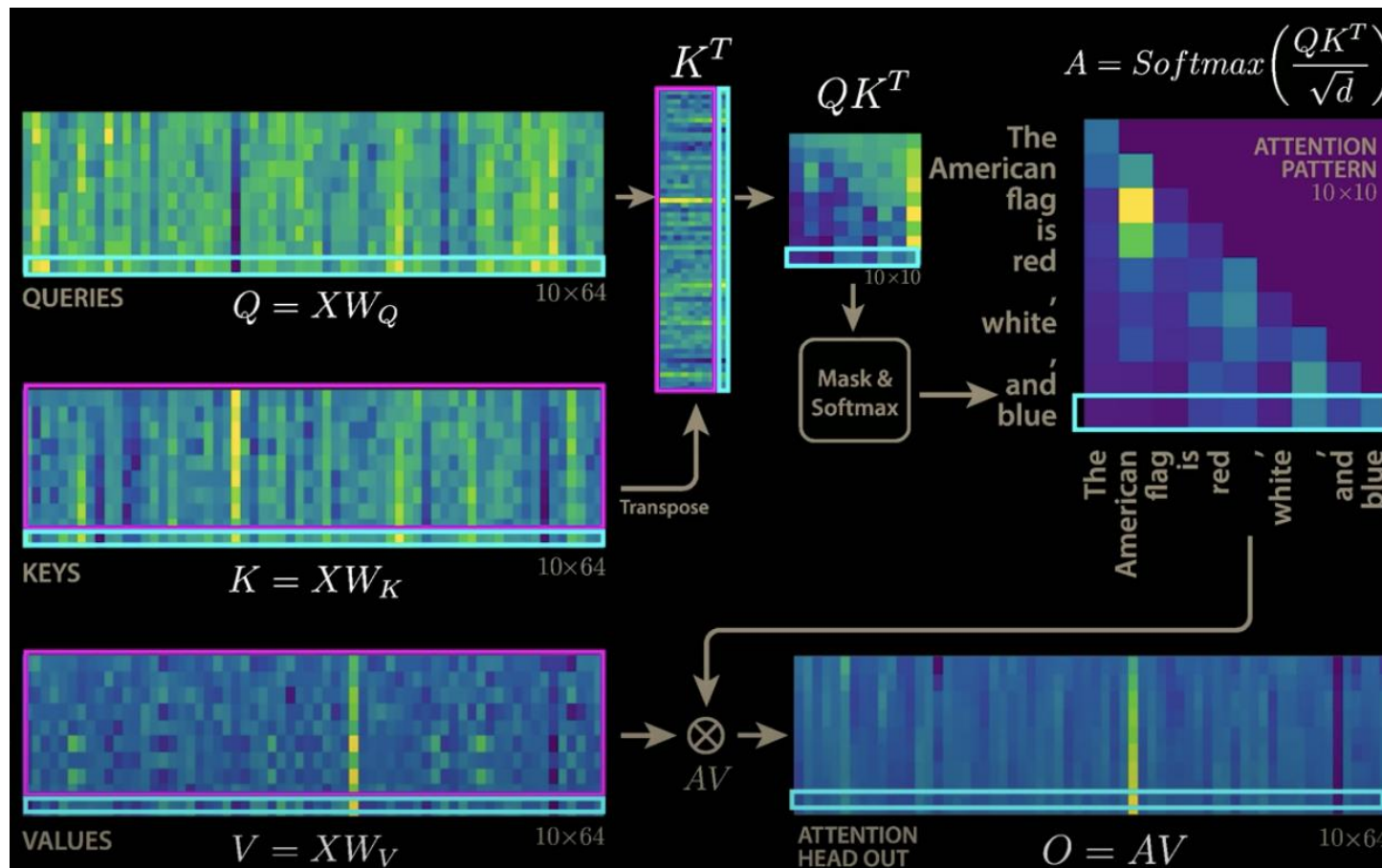
- Store KV values of previous tokens in memory
- When calculating the attention scores for a new token, we retrieve old KV matrices and concatenate to the new k and v vector



Autoregressive Generation Introduces KV Ccache



- Store KV values of previous tokens in memory
- When calculating the attention scores for a new token, we retrieve old KV matrices and concatenate to the new k and v vector
- Faster and more efficient generation at the cost of **increased memory usage**



Question: why there is no such Q cache?

KV Cache makes generation I/O-intensive



Let us implement attention block: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

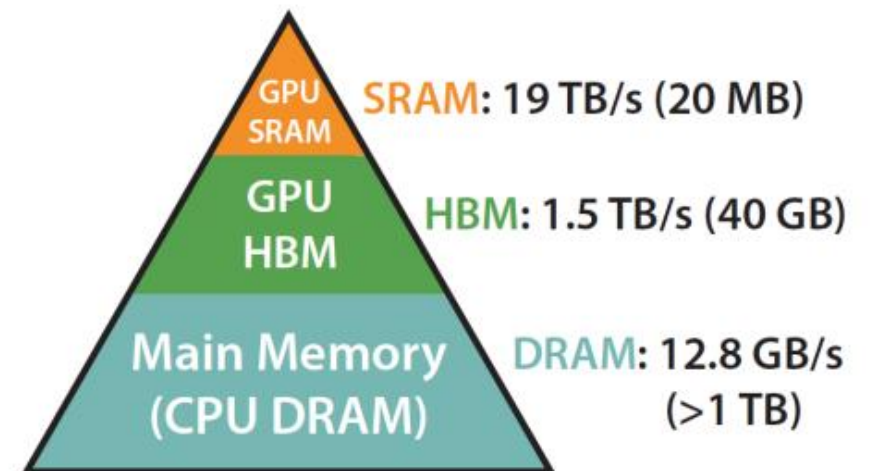
Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

- 1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{QK}^T$, write $\mathbf{S}$ to HBM.
 - 2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \text{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
 - 3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{PV}$, write $\mathbf{O}$ to HBM.
 - 4: Return $\mathbf{O}$.
-

Memory is arranged hierarchically

- GPU SRAM is small, and supports the fastest access
- GPU HBM is larger but with much slower access
- CPU DRAM is huge, but the slowest of all



Memory Hierarchy with
Bandwidth & Memory Size

KV Cache makes generation I/O-intensive



Let us implement attention block: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

Algorithm 0 Standard Attention Implementation

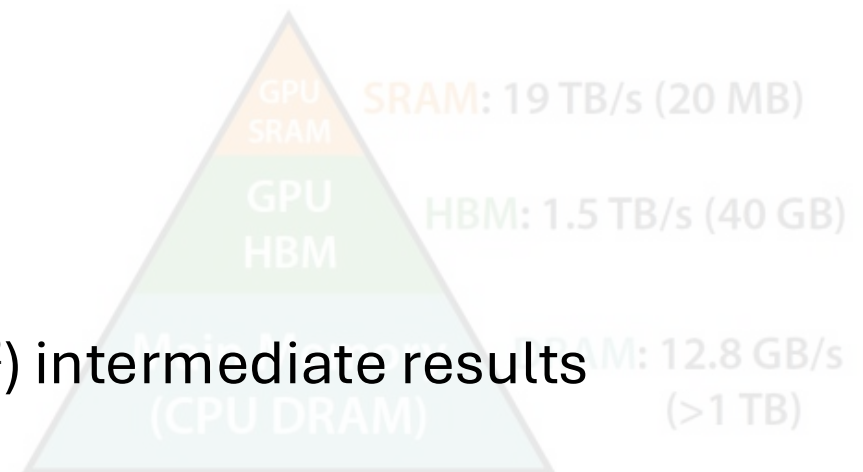
Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

- 1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{QK}^T$, write $\mathbf{S}$ to HBM.
 - 2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \text{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
 - 3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{PV}$, write $\mathbf{O}$ to HBM.
 - 4: Return $\mathbf{O}$.
-

Memory is arranged hierarchically

Challenges: GPU HBM is small, and supports the fastest

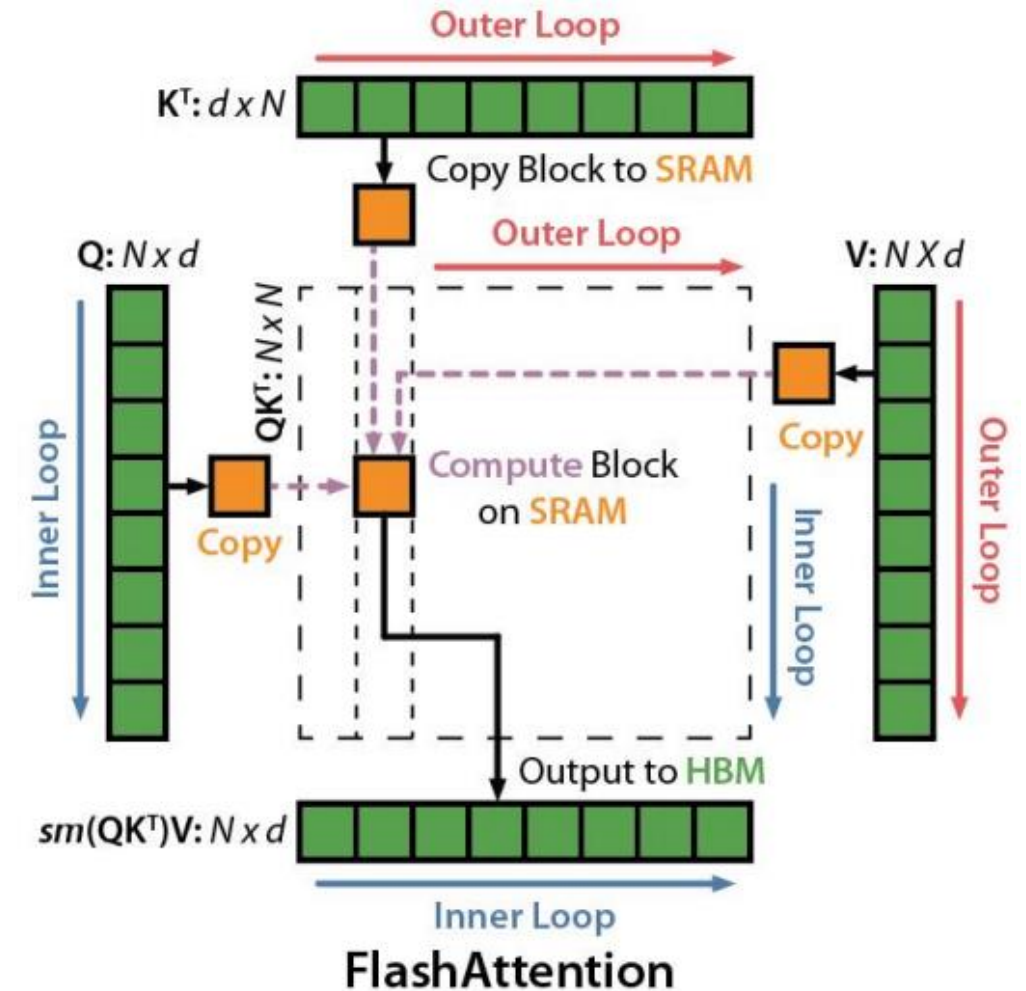
- **Repeated** reads/writes from GPU HBM
- Large **intermediate** results much slower
- Hardly scale to long sequences due to $O(N^2)$ intermediate results
- CPU DRAM is huge, but the slowest of all



Memory Hierarchy with
Bandwidth & Memory Size

- Two key ideas:
 - **Tiling**: compute the attention block by block so that we don't have to load everything into SRAM at once
 - **Operator fusion**: Use a single kernel that includes all operators during computation to avoid kernel launching overhead and intermediate data movement

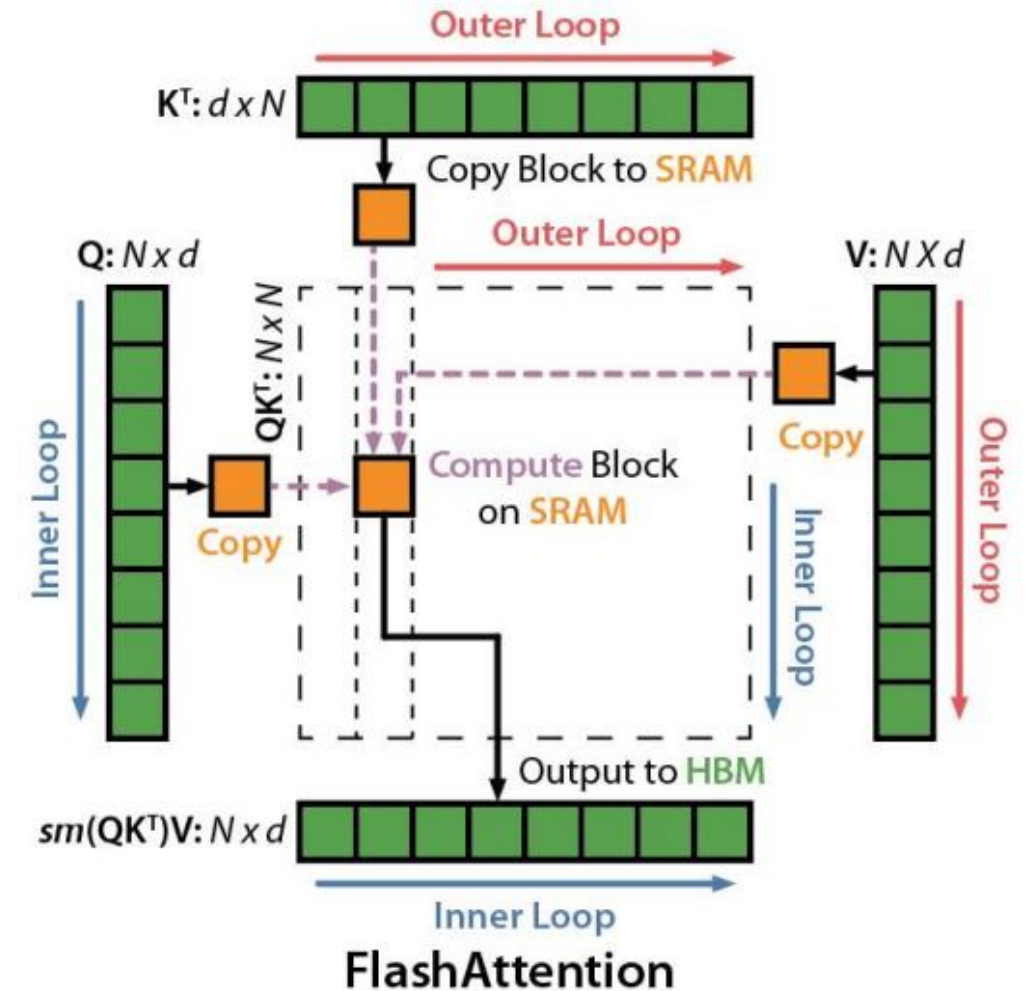
Tiling: Decompose Attention Calculation into Smaller Blocks



Tiling: Decompose Attention Calculation into Smaller Blocks



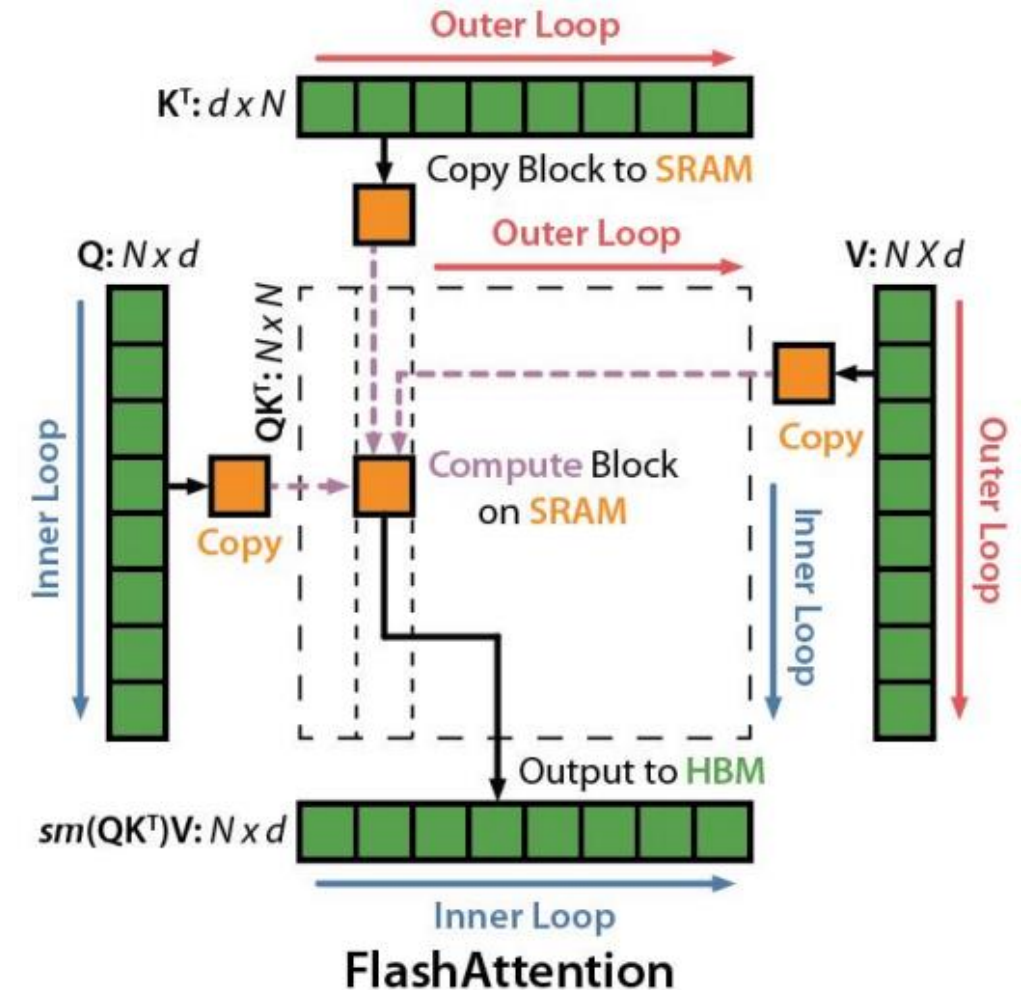
1. Load inputs **by blocks** from global HBM to SRAM



Tiling: Decompose Attention Calculation into Smaller Blocks



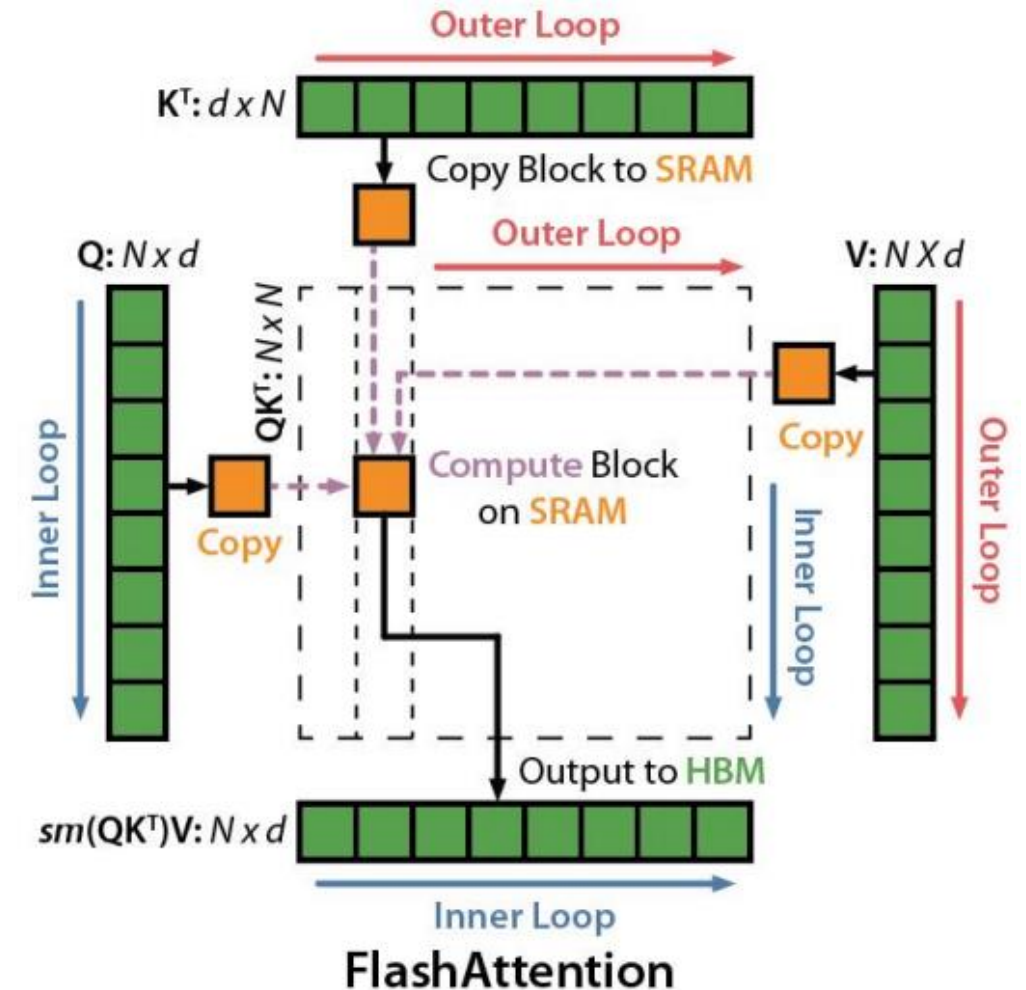
1. Load inputs **by blocks** from global HBM to SRAM
2. On chip, compute attention output wrt the **block**



Tiling: Decompose Attention Calculation into Smaller Blocks



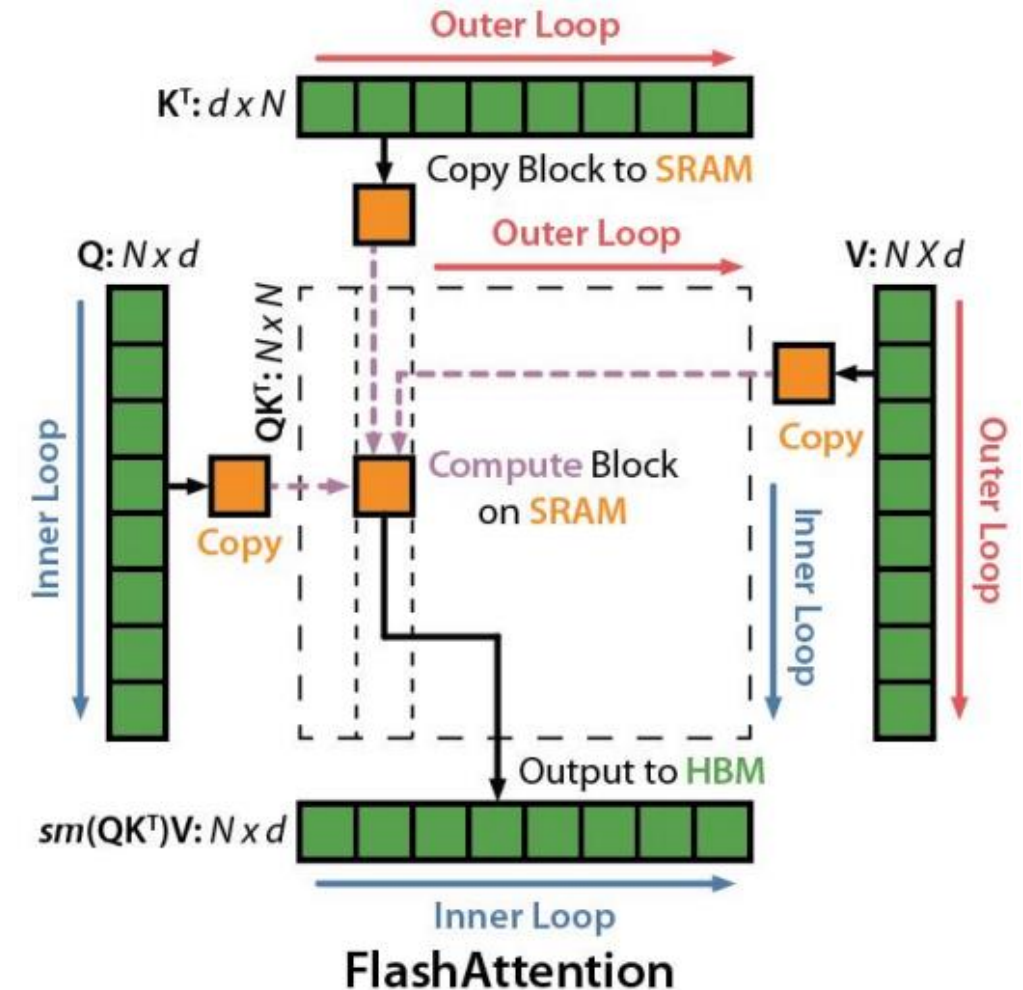
1. Load inputs **by blocks** from global HBM to SRAM
2. On chip, compute attention output wrt the **block**
3. Update output in HBM **by blocks**



Tiling: Decompose Attention Calculation into Smaller Blocks



1. Load inputs **by blocks** from global HBM to SRAM
 2. On chip, compute attention output wrt the **block**
 3. Update output in HBM **by blocks**
- (Everything in a single kernel)



Softmax introduces challenges: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

For a vector $x \in \mathbb{R}^B$, softmax is computed as:

$$\text{softmax}(x) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$



Problem: Exponentials can explode

e^x grows very quickly

Assume $x = 1000$, $e^{1000} \approx 10^{434}$

Too large to store in FP32, overflow (INF)

Subtracting the max value from the input vector before applying the exp function, which helps prevent overflow issue

1. Compute the maximum value in x :

$$m(x) := \max_i x_i$$

Subtracting the max value from the input vector before applying the exp function, which helps prevent overflow issue

1. Compute the maximum value in x :

$$m(x) := \max_i x_i$$

2. Compute the adjusted exponentials (**B blocks**):

$$f(x) := \left[e^{x_1 - m(x)}, e^{x_2 - m(x)}, \dots, e^{x_B - m(x)} \right]$$

Subtracting the max value from the input vector before applying the exp function, which helps prevent overflow issue

1. Compute the maximum value in x :

$$m(x) := \max_i x_i$$

2. Compute the adjusted exponentials (**B blocks**):

$$f(x) := [e^{x_1 - m(x)}, e^{x_2 - m(x)}, \dots, e^{x_B - m(x)}]$$

3. Compute the normalization denominator:

$$\ell(x) := \sum_i f(x)_i$$

Subtracting the max value from the input vector before applying the exp function, which helps prevent overflow issue

1. Compute the maximum value in x :

$$m(x) := \max_i x_i$$

2. Compute the adjusted exponentials (**B blocks**):

$$f(x) := [e^{x_1 - m(x)}, e^{x_2 - m(x)}, \dots, e^{x_B - m(x)}]$$

3. Compute the normalization denominator:

$$\ell(x) := \sum_i f(x)_i$$

4. Compute the final softmax:

$$\text{softmax}(x) = \frac{f(x)}{\ell(x)}$$

Subtracting the max value from the input vector before applying the exp function, which helps prevent overflow issue

1. Compute the maximum value in x :

$$m(x) := \max_i x_i$$

But in Attention, we don't know $m(x)$ in advance?

2. Compute the adjusted exponentials (**B blocks**):

$$f(x) := [e^{x_1 - m(x)}, e^{x_2 - m(x)}, \dots, e^{x_B - m(x)}]$$



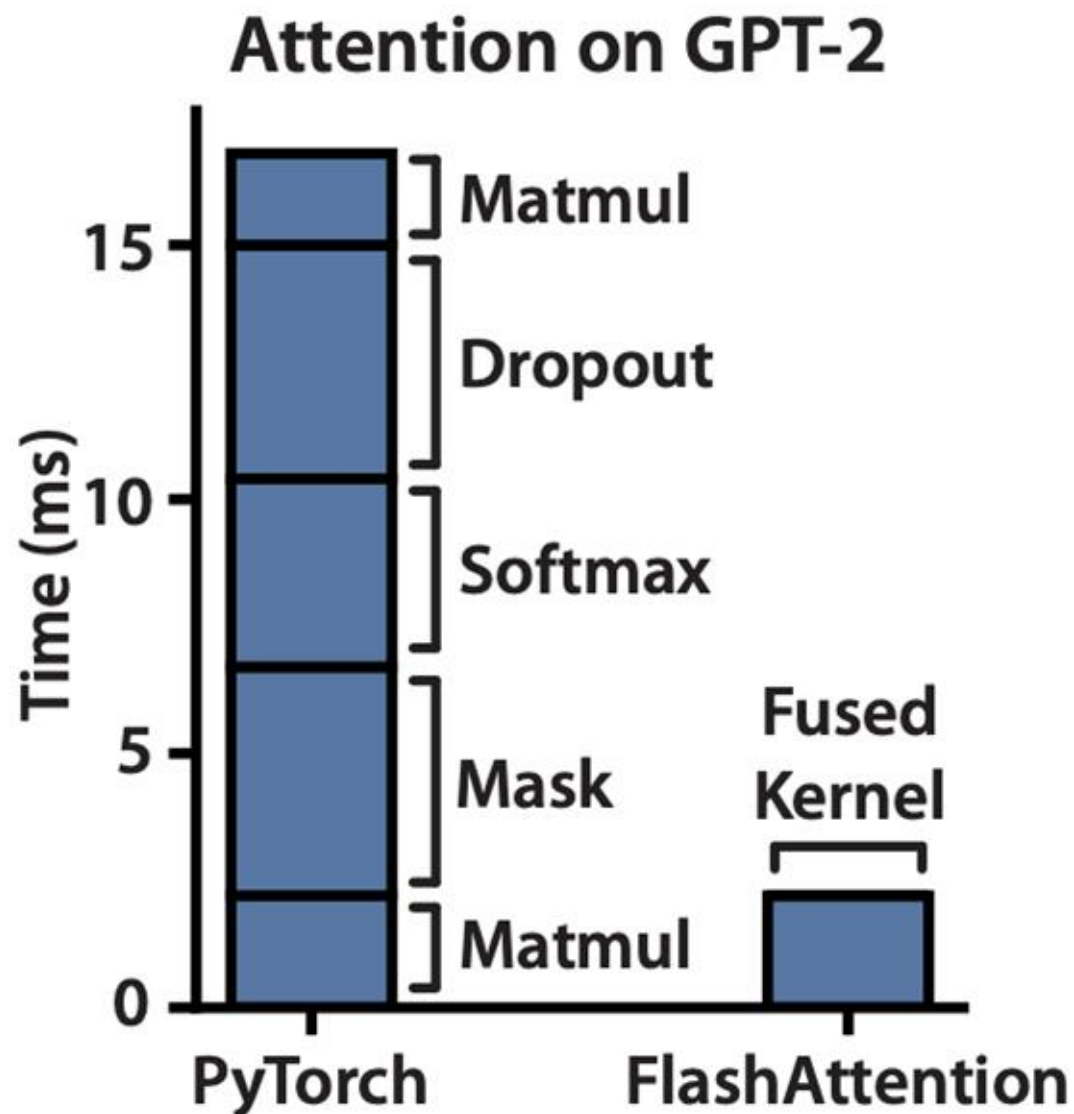
Key Idea: When we find a new max in x , we correct the running sum by multiplying by $e^{\text{old_max} - \text{new_max}}$

3. Compute the normalization denominator:

$$\ell(x) := \sum_i f(x)_i$$

4. Compute the final softmax:

Question: limitations of FlashAttention?



- We introduced the original transformer architecture
 - Tokenization, positional encoding
 - Attention Layer (KV cache)
- FlashAttention achieves speedups by addressing memory bottlenecks in attention
 - Tiling: compute the attention **block by block** with online softmax so that we don't have to load everything into SRAM at once
 - Operator fusion: Use a **single kernel** to avoid kernel launching overhead and intermediate data movement

Team registration (4-5 members in a team by Sept 5)

FlashAttention: Putting it all together

$$S = QK^T \quad A = \text{Softmax}(S) \quad O = AV \quad S = \{x_1, x_2, \dots, x_N\}$$

$$x_i = qk_i^T$$

$$d'_i = d'_{i-1} e^{m_{i-1} - m_i} + e^{x_i - m_i}$$

$$m_0 = -\infty$$

$$d_0 = 0$$

for $1 \leq i \leq N$ **do**

$$x_i = qk_i^T$$

$$m_i = \max(m_{i-1}, x_i)$$

$$d'_i = d'_{i-1} e^{m_{i-1} - m_i} + e^{x_i - m_i}$$

$$o_0 = 0$$

for $1 \leq i \leq N$ **do**

$$a_i = e^{x_i - m_N} / d'_N$$

$$o_i = o_{i-1} + a_i v_i$$

return o_N

$$o'_i = \left(\sum_{j=1}^{i-1} \frac{e^{x_j - m_{i-1}}}{d'_{i-1}} v_j \right) \frac{d'_{i-1}}{d'_i} e^{m_{i-1} - m_i} + \frac{e^{x_i - m_i}}{d'_i} v_i$$

FlashAttention: Putting it all together

$$S = QK^T \quad A = \text{Softmax}(S) \quad O = AV \quad S = \{x_1, x_2, \dots, x_N\}$$
$$x_i = qk_i^T$$

for $1 \leq i \leq N$ **do**

$$x_i = qk_i^T$$

$$m_i = \max(m_{i-1}, x_i)$$

$$d'_i = d'_{i-1} e^{m_{i-1} - m_i} + e^{x_i - m_i}$$

$$o'_i = o'_{i-1} \frac{d'_{i-1}}{d'_i} e^{m_{i-1} - m_i} + \frac{e^{x_i - m_i}}{d'_i} v_i$$

return o'_N

 **FLASH ATTENTION**

Fast and Memory-Efficient Exact Attention with IO-Awareness, 2022